Jeremy Tello

San Antonio, TX | +1 (210) - 501 - 9073 | jeremytello31@gmail.com | LinkedIn | GitHub | Portfolio

Education

Texas A&M University-San Antonio — **San Antonio, TX**
Bachelor's of Science, Computer Science — June 2023 - May 2025

- *GPA: 3.50/4.00, President's List*

Palo Alto College — **San Antonio, TX**
Associate of Science — August 2019 - December 2022

Projects

NFL Stats | GitHub

- Built a scheduled ETL pipeline using Python, Celery, and Redis to automate weekly web scraping of NFL player statistics, ensuring the PostgreSQL database remains synchronized with current season data.
- Built Django REST APIs for NFL stats retrieval with Redis-backed caching, rate limiting, and tightened CORS/CSRF protections, then paired them with TanStack Query on the frontend to reduce redundant fetches and improve data responsiveness.
- Containerized using Docker and configured Nginx as a reverse proxy to serve static JavaScript React assets and route traffic, ensuring environment parity between local and production environments.
- Built a GitHub Actions CI/CD pipeline that runs linting, unit tests, mocks, and Trivy container scans before publishing Docker images to Docker Hub, improving release quality and catching container vulnerabilities before deployment.

Portfolio | GitHub

- Provisioned and managed a fully automated cloud architecture on AWS using Terraform to enforce Infrastructure as Code (IaC) and declarative state management for zero-drift deployments.
- Developed a responsive frontend using TypeScript, compiling static assets hosted on AWS S3 and distributed traffic globally via AWS CloudFront (CDN), leveraging AWS Certificate Manager (ACM) for custom SSL/TLS certificate management and HTTPS encryption.
- Engineered a serverless event-driven pipeline using AWS API Gateway and AWS Lambda to validate payloads, dispatch transactional emails via AWS SES, and persist structured message data into AWS DynamoDB.
- Secured cloud infrastructure by enforcing AWS IAM least-privilege policies and CloudFront Origin Access Control (OAC) while routing backend requests to interface safely with a production VPS.

VulGPT | GitHub

- Built a concurrent vulnerability scanning backend using Python, Django, and Neo4j to map and query complex transitive dependency graphs for automated threat analysis.
- Integrated an asynchronous task pipeline using Celery, Redis, and the GitHub API to parse and ingest OSV JSON payloads without blocking the main execution thread or triggering rate-limit throttling.
- Built a React + GraphQL interface on top of a Django/Neo4j backend that ingests dependency manifests, traces transitive vulnerabilities, and uses local Ollama models to summarize CVEs and explain lower-risk upgrade paths.
- Containerized the application stack using Docker and leveraged Flower to monitor task health and worker load.